

AI ASSISTED CODING

NAME: NITHIN

HALL_NO: 2303A51096

Assignment—4

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

Write a Python function that classifies a given text as Spam or Not

Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."

Expected Output:

Spam.

Code with prompt:

```
spam or not.py > ...
1 #write a function to determine if a message is spam or not
2 def is_spam(message):
3     spam_keywords = ["urgent", "free", "win", "cash", "prize", "click here"]
4     for keyword in spam_keywords:
5         if keyword in message.lower():
6             return True
7     return False
8 message = input("Enter the message: ")
9 if is_spam(message):
10     print("The message is spam.")
11 else:
12     print("The message is not spam.")
```

Input and Output:

```
PS C:\Users\nithi\Desktop\ai assisted coding> & C:/Users/nithi/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/nithi/Desktop/ai assisted coding/spam or not.py"
Enter the message: "Congratulations! You have won a free lottery ticket."
The message is spam.
PS C:\Users\nithi\Desktop\ai assisted coding>
```

Q2. One-Shot Prompting (Emotion detection)

Task:

Write a Python program that detects the emotion of a sentence

using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion

Code with prompt:

```

1 shot emotions.py > ...
1  #Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']
2  #display the emotion of the given text
3  #write a python program to detect the emotion of a given text
4  def detect_emotion(text):
5      happy_words = ['happy', 'joy', 'pleased', 'delighted', 'content']
6      sad_words = ['sad', 'unhappy', 'sorrowful', 'dejected', 'downcast']
7      angry_words = ['angry', 'mad', 'furious', 'irate', 'livid']
8      excited_words = ['excited', 'thrilled', 'elated', 'eager', 'enthusiastic']
9      nervous_words = ['nervous', 'anxious', 'tense', 'apprehensive', 'worried']
10
11     text = text.lower()
12     if any(word in text for word in happy_words):
13         return 'happy'
14     elif any(word in text for word in sad_words):
15         return 'sad'
16     elif any(word in text for word in angry_words):
17         return 'angry'
18     elif any(word in text for word in excited_words):
19         return 'excited'
20     elif any(word in text for word in nervous_words):
21         return 'nervous'
22     else:
23         return 'neutral'
24 input_text = input("Enter a text: ")
25 emotion = detect_emotion(input_text)
26 print(f"The emotion of the given text is: {emotion}")

```

Output:

```

Enter a text: i have injured while cutting vegetables
The emotion of the given text is: neutral
PS C:\Users\nithi\Desktop\ai assisted coding> & C:/Users/nithi/AppData/Local/Programs/Python/Python312/python.exe "c:\
shot emotions.py"
Enter a text: i am very excited to watch movie
The emotion of the given text is: excited
PS C:\Users\nithi\Desktop\ai assisted coding>

```

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

Write a Python program that predicts a student's grade based on

marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F

Code:

```
lab-6 > lab_4 > grades_few_shot.py > predict_grade
1  def predict_grade(marks):
2      if marks >= 90:
3          return 'A'
4      elif marks >= 80:
5          return 'B'
6      elif marks >= 70:
7          return 'C'
8      elif marks >= 60:
9          return 'D'
10     else:
11         return 'F'
12
13     print(predict_grade(92))
14     print(predict_grade(82))
15
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL ...
PS C:\Users\nithi\Desktop\ai assisted coding> & C:/
Users/nithi/AppData/Local/Programs/Python/Python312
/python.exe "c:/Users/nithi/Desktop/ai assisted cod
ing/lab-6/lab_4/grades_few_shot.py"
A
B
PS C:\Users\nithi\Desktop\ai assisted coding>
```

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign

(Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model):
The Indian

Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meena

Code:

```
lab-6 > lab_4 > rashi.py > ...
1  # Input: March
2  # Output: Mesha
3  # Input: august
4  # Output: Kanya
5  # Input: January
6  # Output: Kumbha
7  # Generate clean, readable, and optimized Python code.
8  def get_rashi(month):
9      rashi_map = {
10         'january': 'Kumbha', 'february': 'Meena', 'march': 'Mesha', 'april': 'Vrishabha',
11         'may': 'Mithuna', 'june': 'Karka', 'july': 'Simha', 'august': 'Kanya', 'september': 'Tula',
12         'october': 'Vrishchika', 'november': 'Dhanu', 'december': 'Makara'}
13     month = month.lower()
14     return rashi_map.get(month, "Invalid month")
15 month = input("Enter the month: ")
16 rashi = get_rashi(month)
17 print(f"The Rashi for {month.capitalize()} is: {rashi}")
```

Input:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\nithi\Desktop\ai assisted coding> & C:/Users/nithi/AppData/Local/Programs/Python/Python312/p
ab-6/lab_4/rashi.py
Enter the month: january
The Rashi for January is: Kumbha
PS C:\Users\nithi\Desktop\ai assisted coding> |
```

Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student

Passes or Fails based on marks using Chain-of-Thought (CoT)

prompting.

Result Categories:

['Pass', 'Fail']

code:

```
1 # If marks = 80 → marks ≥ 35 → Pass
2 # If marks = 20 → marks < 35 → Fail
3
4 # Generate an optimized Python program following this step-by-step reasoning.
5 def predict_grade(marks):
6     if marks >= 35:
7         return 'pass'
8     else:
9         return 'fail'
10 marks = int(input("Enter the marks: "))
11 result = predict_grade(marks)
12 print(f"The result is: {result}")
13
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Enter the month: january
The Rashi for January is: Kumbha
PS C:\Users\nithi\Desktop\ai assisted coding> & C:/Users/nithi/AppData/Local/Programs/Python/Python310/python.exe C:/Users/nithi/Desktop/ai assisted coding/lab-6/lab_4/marks.py
Enter the marks: 55
The result is: pass
PS C:\Users\nithi\Desktop\ai assisted coding> 
```

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting code:

```
lab-6 > lab_4 > vote.py > ...
1  # Write a Python program that determines whether a person is
2  # eligible to vote using Chain-of-Thought (CoT) prompting
3  def is_eligible_to_vote(age):
4      if age >= 18:
5          return "Eligible to vote"
6      else:
7          return "Not eligible to vote"
8
9  age = int(input("Enter your age: "))
10 eligibility = is_eligible_to_vote(age)
11 print(f"You are {eligibility}.")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Enter your age: 22
You are Eligible to vote.
PS C:\Users\nithi\Desktop\ai assisted coding> & C:/Users/nithi/AppData/Local/Programs/Python/Python312/python.exe
ab-6/lab_4/vote.py
Enter your age: 17
You are Not eligible to vote.
PS C:\Users\nithi\Desktop\ai assisted coding> 
```